

Top .NET Interview Questions

A curated guide covering C#, .NET Core/CLR, OOP, async programming, and ASP.NET Core

Core .NET & CLR Concepts

1. What is the .NET CLR (Common Language Runtime)?

The CLR is the runtime engine of .NET that manages code execution. It handles memory management (garbage collection), type safety, exception handling, thread management, and JIT (Just-In-Time) compilation of Intermediate Language (IL) code into native machine code.

2. What is the difference between .NET Framework, .NET Core, and .NET 5+?

.NET Framework is the original Windows-only runtime. .NET Core was a cross-platform, open-source rewrite. Starting with .NET 5, Microsoft unified everything into a single platform simply called '.NET', released annually, that is cross-platform, high-performance, and open source.

3. What is IL (Intermediate Language) and JIT compilation?

When C# code is compiled, it produces IL (also called MSIL/CIL), a CPU-independent set of instructions. At runtime, the JIT compiler converts IL into native machine code specific to the host platform, right before it is executed.

4. What is the difference between managed and unmanaged code?

Managed code runs under the control of the CLR, which provides services like garbage collection and type safety. Unmanaged code (e.g., C/C++ binaries) runs directly on the OS without CLR oversight, so the developer must manage memory manually.

5. What is an assembly in .NET?

An assembly is the compiled output of a .NET project (a .dll or .exe). It's the fundamental unit of deployment, versioning, and security, containing IL code, metadata, and a manifest describing its contents.

C# Language Fundamentals

1. What is the difference between value types and reference types?

Value types (struct, int, bool, enum) store data directly on the stack and are copied by value when assigned. Reference types (class, string, array, interface) store a reference on the stack that points to the actual data on the heap, so assignment copies the reference, not the data.

2. What is boxing and unboxing?

Boxing is converting a value type into a reference type by wrapping it in an object on the heap. Unboxing is the reverse: extracting the value type from the object. Both operations carry a performance cost and should be minimized in hot code paths.

3. What is the difference between == and .Equals()?

For reference types, == compares references by default (unless overridden), while .Equals() can be overridden to compare logical/value equality. For value types, == and .Equals() typically both compare values. Strings override == to compare content rather than reference.

4. What are nullable types and the null-conditional operator?

Nullable value types (e.g., `int?`) allow value types to also hold null, implemented via the generic `Nullable` struct. The null-conditional operator (`?.`) lets you safely call members on a possibly-null object, short-circuiting to null instead of throwing a `NullReferenceException`.

5. What is the difference between abstract classes and interfaces?

An abstract class can contain implementation, fields, and constructors, and a class can inherit from only one abstract class. An interface (traditionally) defines only a contract of members without implementation, and a class can implement multiple interfaces. Modern C# allows default interface implementations, blurring this line somewhat.

6. What are delegates and events?

A delegate is a type-safe function pointer that references a method with a matching signature, enabling methods to be passed as parameters. An event is a wrapper built on top of a delegate that restricts external code to only subscribing or unsubscribing, following the publisher-subscriber pattern.

7. What is the difference between 'ref', 'out', and 'in' parameters?

'ref' passes a variable by reference and requires it to be initialized before the call. 'out' also passes by reference but does not require prior initialization, and must be assigned inside the method. 'in' passes by reference as read-only, preventing the method from modifying the argument.

8. What are generics and why are they useful?

Generics allow you to define classes, methods, and interfaces with a placeholder for the data type, specified at instantiation (e.g., `List`). They provide type safety at compile time, eliminate boxing/unboxing for value types, and promote code reuse without duplicating logic per type.

9. What is the difference between 'is' and 'as' operators?

'is' checks whether an object is compatible with a given type and returns a boolean (and can pattern-match into a variable). 'as' attempts a conversion to a given reference or nullable type and returns null if the conversion fails, instead of throwing an exception.

Object-Oriented Programming

1. What are the four pillars of OOP?

Encapsulation (bundling data and methods, restricting direct access), Abstraction (exposing only essential features while hiding implementation details), Inheritance (deriving new classes from existing ones to reuse behavior), and Polymorphism (allowing objects of different types to be treated through a common interface, with behavior resolved at compile or run time).

2. What is method overloading vs. method overriding?

Overloading defines multiple methods with the same name but different parameter lists within the same class (compile-time polymorphism). Overriding lets a derived class provide its own implementation of a virtual/abstract method inherited from a base class (runtime polymorphism), using the 'override' keyword.

3. What is the difference between 'virtual', 'override', and 'new' keywords?

'virtual' marks a base class method as overridable. 'override' provides a new implementation in a derived class that replaces the base version, preserving polymorphic behavior. 'new' hides a base member instead of overriding it, so the method that runs depends on the compile-time type of the reference, not the runtime type.

4. What is composition over inheritance?

It's a design principle favoring building complex behavior by combining (composing) smaller, independent objects rather than relying on deep inheritance hierarchies. This reduces tight coupling and the fragile base class problem, and increases flexibility to change behavior at runtime.

Memory Management & Garbage Collection

1. How does garbage collection work in .NET?

The GC automatically reclaims memory occupied by objects no longer reachable from application roots. .NET uses a generational GC (Gen 0, 1, 2) based on the observation that most objects die young; Gen 0 is collected most frequently and cheaply, while Gen 2 (long-lived objects) is collected less often.

2. What is the difference between the stack and the heap?

The stack stores value types and method call frames (local variables, return addresses); it is fast, has a fixed size, and is automatically cleaned up when a method returns. The heap stores reference type instances, has a larger and more flexible size, and is managed by the garbage collector.

3. What is IDisposable and the using statement used for?

IDisposable defines a Dispose() method for releasing unmanaged resources (file handles, database connections, network sockets) deterministically, rather than waiting for the GC. The 'using' statement (or declaration) ensures Dispose() is called automatically once the object goes out of scope, even if an exception occurs.

4. What are finalizers and when should you use them?

A finalizer (~ClassName()) is called by the GC before reclaiming an object's memory, giving a last chance to release unmanaged resources. They should be used sparingly, since they add overhead and delay collection; the recommended pattern is to implement IDisposable and only add a finalizer as a safety net for unmanaged resources.

Asynchronous Programming & Threading

1. What is the difference between synchronous and asynchronous programming?

Synchronous code executes sequentially, blocking the calling thread until an operation completes. Asynchronous code allows an operation (like I/O) to run without blocking the calling thread, freeing it up to do other work, and resumes execution when the operation finishes.

2. How do async and await work in C#?

'async' marks a method as asynchronous, allowing it to use 'await'. 'await' suspends execution of the method until the awaited Task completes, without blocking the calling thread; control returns to the caller in the meantime, and the rest of the method runs as a continuation once the awaited operation finishes.

3. What is the difference between Task and Thread?

A Thread represents an actual OS-level thread of execution and is relatively expensive to create. A Task represents an asynchronous operation that may or may not use a dedicated thread; tasks are managed by the thread pool, making them lightweight and better suited for scalable, I/O-bound or CPU-bound asynchronous work.

4. What is a deadlock and how can it be avoided?

A deadlock occurs when two or more threads are blocked forever, each waiting on a resource held by the other. It can be avoided by acquiring locks in a consistent order, minimizing lock scope, avoiding blocking calls on async code (like `.Result` or `.Wait()`), and using higher-level constructs like `async/await` consistently.

5. What is the difference between lock, Monitor, and Mutex?

'lock' is C# syntactic sugar around `Monitor.Enter/Exit`, providing mutual exclusion within a single process. `Monitor` offers more control (e.g., `TryEnter`, `Wait`, `Pulse`). `Mutex` works similarly but can be used for synchronization across process boundaries, at a higher performance cost.

ASP.NET Core & Web Development

1. What is dependency injection and how does ASP.NET Core support it?

Dependency injection is a design pattern where an object's dependencies are provided externally rather than created internally, promoting loose coupling and testability. ASP.NET Core has a built-in DI container where services are registered with a lifetime (`Transient`, `Scoped`, or `Singleton`) and injected into constructors automatically.

2. What is middleware in ASP.NET Core?

Middleware are components assembled into a request processing pipeline, each handling requests and responses and optionally passing control to the next component. Examples include authentication, routing, exception handling, and static file serving, configured in the `Program.cs/Startup.cs` pipeline.

3. What is the difference between Transient, Scoped, and Singleton lifetimes?

`Transient` services are created every time they're requested. `Scoped` services are created once per client request (or scope) and reused within that scope. `Singleton` services are created once for the entire application lifetime and shared across all requests.

4. What is the difference between MVC and Web API in ASP.NET Core?

In ASP.NET Core, MVC and Web API have been unified into a single framework. Controllers can return views (HTML, for traditional MVC apps) or serialized data like JSON (for APIs), typically using the same base `'Controller'` or lightweight `'ControllerBase'` class for APIs.

5. What is Entity Framework Core and what is the difference between eager and lazy loading?

Entity Framework Core is an object-relational mapper (ORM) that lets developers work with a database using .NET objects instead of raw SQL. Eager loading retrieves related data upfront in the same query (using `Include()`), while lazy loading defers loading related data until it is actually accessed, issuing additional queries on demand.

LINQ & Collections

1. What is LINQ and what are its benefits?

LINQ (Language Integrated Query) provides a consistent, declarative syntax for querying collections, databases, XML, and other data sources directly within C#. Benefits include compile-time type checking, IntelliSense support, and a unified query syntax across very different data sources.

2. What is the difference between IEnumerable and IQueryable?

IEnumerable executes queries in memory and is best for querying in-memory collections like lists or arrays. IQueryable builds an expression tree that is translated into a query (e.g., SQL) and executed at the data source, making it more efficient for querying remote data sources like databases.

3. What is deferred execution in LINQ?

Deferred execution means a LINQ query is not actually run when it is defined, but only when the results are iterated over (e.g., in a foreach loop or when calling ToList()). This allows query composition and ensures the query reflects the latest state of the data source at execution time.

Exception Handling & Best Practices

1. What is the difference between 'throw' and 'throw ex'?

'throw' re-throws the current exception while preserving the original stack trace, which is important for debugging. 'throw ex' resets the stack trace to the point of the re-throw, losing information about where the exception originally occurred, and is generally discouraged.

2. What is the purpose of a finally block?

A finally block contains code that always executes after a try block, whether or not an exception was thrown or caught, making it ideal for cleanup logic like closing files or releasing resources that must run regardless of the outcome.

3. What are custom exceptions and when should you create one?

Custom exceptions are user-defined classes deriving from Exception (or a more specific built-in exception) that represent domain-specific error conditions. They should be created when built-in exception types don't adequately describe the error, helping callers catch and handle specific failure scenarios more precisely.